# ■ Docker Notes

A CompleteReferenceGuide·FromZerotoCompose

---

## 1. What is Docker?

Dockerisatool usedtopackagean application with everything it needs to run:

- Application Code
- Dependencies
- Runtime
- Environment Variables
- System Libraries
- Configuration

> ■ Instead of "It works on my machine" — Docker makes it "It works the same everywhere."

## 2. Why Docker Exists

| Without Docker | With Docker |
|---|---|
| Dev: Node 20, MongoDB installed, works fine | Docker Image: Node version fixed |
| Server: Node 18, MongoDB missing → App crashes | Dependencies fixed, config fixed, runs everywhere |

## 3. Docker Architecture

```
Your Terminal
    |
    v
Docker CLI
    |
    v
Docker Daemon
    |
    v
Images / Containers / Networks / Volumes
```

**Example:** `dockerrun nginx`

CLI sends command → Docker Daemon → pulls image → creates container → runs app

## 4. Important Docker Terms

### Image

A blueprint/template. You cannot run an image directly — Docker creates a container from it. Image = Class | Container = Object

### Container

A running instance of an image. One image can create many containers.

## Dockerfile

Arecipetocreate an image.

## Docker Compose

Runs multiplecontainerstogether (frontend, backend, DB). Instead of running 10 long commands, you write one docker-compose.yml.

## Namespace

Isolatescontainers — makes each container think it has its own process list, network, filesystem, users, and hostname.

## Cgroups

Controlresource limits — how much CPU, RAM, Disk I/O, and Network a container can use.

## Volume

Stores persistent data. Containers are temporary — if you delete one, internal data is gone. Volumes survive container deletion.

## Network

HowDockercontainers communicate. Inside Compose, use the service name (e.g. mongo), not localhost.

## Port Mapping

Mapshostportstocontainer ports. Format: HOST_PORT:CONTAINER_PORT Example: -p 3000:5173

## .dockerignore

TellsDockerwhatNOT to copy into the image (node_modules, dist, .git, .env, etc.)

## Docker Cache

Dockerbuildsimages layer by layer and caches each layer. Copy package.json first, then run npm install, then copy source — so code changes don't bust the install cache.

## Dockerfile Example

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

## 5. How Docker Runs an App

```
Dockerfile
   |
   v
docker build
   |
   v
Image
   |
   v
docker run
   |
   v
Container
   |
   v
Running App
```

```
docker build -t my-react-app .
docker run -p 5173:5173 my-react-app
```

## 6. Installing Docker (Mac)

- Download Docker Desktop forMac
- Install it and open Docker Desktop
- Wait until Docker Engine starts

```
docker        --version
docker compose version
docker run hello-world
```

## 7. React Vite Quick Start

```
npm createvite@latest  react-docker
cd react-docker
npm install
npm run dev
```

- Vite normally runs on http://localhost:5173

## 8. Dockerfile for React Vite

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173
```

```
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

| Instruction | What it does |
|---|---|
| `FROM node:20-alpine` | Node 20 on lightweight Linux |
| `WORKDIR /app` | Creates & moves into /app inside container |
| `COPY package*.json ./` | Copies package files first (cache optimization) |
| `RUN npm install` | Installs dependencies inside the image |
| `COPY . .` | Copies the rest of your project |
| `EXPOSE 5173` | Documents the port the app uses |
| `CMD [...]` | Runs Vite exposed outside the container |

■ Without --host 0.0.0.0, Vite only listens inside the container and your browser will refuse connection.

## 9. .dockerignore

```
node_modules
dist
.git
.gitignore
Dockerfile
docker-compose.yml
.env
npm-debug.log
```

■ Without .dockerignore your image becomes bloated. Copying node_modules from your Mac into a Linux container is a classic beginner mistake.

## 10–21. Essential Docker Commands

### Build the Image

```
docker build -t react-docker .
```

### Run the Container

```
docker run -p 5173:5173 react-docker
```

### Run with Name

```
docker run --name react-container -p 5173:5173 react-docker
```

### Run Detached

```
docker run -d --name react-container -p 5173:5173 react-docker
```

### See Running Containers

```
docker ps
docker ps -a
```

### Stop / Start / Remove

```
docker stop react-container
docker start react-container
docker rm react-container
```

```
docker rm -f react-container
```

### Images

```
docker images
docker rmi react-docker
docker rmi -f react-docker
```

### Logs

```
docker logs react-container
docker logs -f react-container
```

### Shell into Container

```
docker exec -it react-container sh
```

## 22–24. Docker Compose

```
services:
  client:
    build: .
    container_name: react_client
    ports:
      - "5173:5173"
    volumes:
      - .:/app
      - /app/node_modules
    stdin_open: true
    tty: true
```

| Command | Description |
| --- | --- |
| `docker compose up` | Start services |
| `docker compose up -d` | Start in background |
| `docker compose up --build` | Rebuild and start |
| `docker compose down` | Stop and remove containers |
| `docker compose down -v` | Stop + remove volumes |
| `docker compose down --rmi all -v` | Stop + remove images + volumes |
| `docker compose logs -f` | Follow logs |
| `docker compose ps` | List compose containers |
| `docker compose exec client sh` | Shell into a service |

## 25–26. Full MERN Docker Compose

```
services:
  client:
    build: ./client
    container_name: client_c
    ports:
      - "5173:5173"
    volumes:
      - ./client:/app
```

```
      - /app/node_modules
    depends_on:
      -api

  api:
    build: ./api
    container_name: api_c
    ports:
      - "4000:4000"
    volumes:
      - ./api:/app
      - /app/node_modules
    environment:
      - MONGO_URI=mongodb://mongo:27017/mydb
    depends_on:
      - mongo

  mongo:
    image: mongo
    container_name: mongo_c
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

volumes:
  mongo-data:
```

### How Containers Talk

InsideDockerCompose,services use service names as hostnames:

```
# CORRECT
mongoose.connect("mongodb://mongo:27017/mydb");

# WRONG (localhost means the same container)
mongoose.connect("mongodb://localhost:27017/mydb");
```

## 27–28. Image Layers & Command Reference

### Image Layers

```
FROM node       → layer 1
WORKDIR /app    → layer 2
COPY package.json → layer 3
RUN npminstall  → layer 4  (cached if package.json unchanged)
COPY sourcecode   → layer 5
CMD npmrundev   → layer 6
```

### Full Command Reference

| Category | Command |
| --- | --- |
| Images | `docker images` |
| Images | `docker build -t app-name .` |

| | |
|---|---|
| Images | `docker rmi image-name` |
| Images | `docker image prune` |
| Containers | `docker ps / docker ps -a` |
| Containers | `docker run image-name` |
| Containers | `docker stop / start / restart container` |
| Containers | `docker rm container-name` |
| Containers | `docker logs container-name` |
| Containers | `docker exec -it container sh` |
| Cleanup | `docker system prune` |
| Cleanup | `docker system prune -a` |
| Cleanup | `docker system prune -a --volumes` |

## 29. Common Beginner Errors

### ■ localhost refused to connect

**Cause:** Vite only listening inside container.

**Fix:**

```
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

### ■ package.json not found

**Cause:** Wrong folder mounted in volume.

**Fix:**

```
# Make sure the path points to the folder with package.json
-v /Users/you/Desktop/my-app:/app
```

### ■ port already in use

**Cause:**Somethingelseisusing the port.

**Fix:**

```
lsof -i :5173
kill -9 PID
# or use a different port:
docker run -p 3000:5173 react-docker
```

## 30. Docker Flow Summary

- 1. Write appcode
- 2. Write Dockerfile
- 3. Write .dockerignore
- 4. Build image
- 5. Run container
- 6. Map ports
- 7. Use volumes for live updates
- 8. Use Compose for multiple services

## 31. Best Mental Model

| Concept | Docker Equivalent |
|---|---|
| Recipe | Dockerfile |
| Cooked meal packed in a box | Image |
| Meal being served | Container |
| Restaurant order for multiple meals | Docker Compose |
| Storage box that survives | Volume |
| Private road between containers | Network |
| Door between your machine and container | Port |

## 32. Minimum Commands You Must Know

```
docker build -t app.
docker run -p 5173:5173 app
docker ps
docker ps -a
docker stop container
docker rm container
docker images
docker rmi image
docker logs container
docker exec -it container sh
docker compose up
docker compose up --build
docker compose down
docker system prune -a --volumes
```